QALight

Frontend-розробка

Заняття 14. Робота із масивами

Викладач курсу: Іван Ніконов, ТОВ «Тренінг-центр КьюЕйЛайт»

Порядок денний



Робота із масивами

Методи і властивоті масивів, цикли.

Створення програми сортування масиву чисел

Робота із масивами

Методи і властивоті масивів, цикли.

Масиви JavaScript

Теорія: Масиви JavaScript

Об’єкт **Array** (масив) дозволяє зберігати **колекцію** з кількох елементів за допомогою однієї змінної.

У JavaScript масиви **є** об’єктами з такими основними характеристиками:

- Масиви мають змінний розмір і можуть містити **суміш різних типів** даних.
- Масиви **не є асоціативними масивами**, тому до елементів масиву не можна отримати доступ за допомогою **ключів,** лише за допомогою невід’ємних цілих чисел (**індексів**).
- Перший елемент має індекс 0, другий - 1 і так далі, а останній елемент - **length - 1**.
- Операції копіювання масивів створюють **поверхневі копії**. (Усі стандартні вбудовані операції копіювання з будь-якими об’єктами JavaScript створюють поверхневі копії, а не глибокі).

Існує три способи створення нового масиву:

- за допомогою нотації літералів масиву: `let empty = [];`
- за допомогою конструктора `Array()`: `var arr = new Array();`
- за допомогою `split()` для створення масиву з рядка: `const fruits = 'Apple, Banana'.split(', ');`

Масив – це впорядкована колекція

Теорія: Масиви JavaScript

Масиви зазвичай описуються як "**спискові об'єкти**"; в основному це окремі об’єкти, які містять кілька значень, збережених у списку. Об’єкти-масиви можна зберігати в змінних і працювати з ними так само, як і з будь-яким іншим типом значень, але різниця полягає в тому, що ми можемо отримати доступ до кожного значення всередині списку окремо та виконувати зі списком циклічні операції для кожного значення списку.

Масив – це **впорядкована колекція** значень.

```
console.log(typeof []); // "object"
```

Значення колекції називаються **елементами**. Кожний елемент масиву має свій числовой порядковий номер, що називається **індексом**. Індекси починаються з 0.

Масив може містити у собі не одне значення, причому значення можуть мати різні типи даних.

```
let customArr = [6, true, 'Apple'];
```

Створення масиву

Теорія: Масиви JavaScript

Існує три способи створення нового масиву:

- за допомогою нотації літералів масиву: let empty = [];
- за допомогою конструктора Array(): var arr = new Array();
- за допомогою split() для створення масиву з рядка: const fruits = 'Apple, Banana'.split(', ');

Метод **array.fill(value[, fromIndex[, toIndex]])** заповнює масив значенням, починаючи **fromIndex** до **toIndex** (за винятком самого toIndex). Додатковий аргумент fromIndex за замовчуванням дорівнює 0, необов'язковий аргумент **toIndex** за замовчуванням має значення **array.length**.

```
const numbers = [1, 2, 3, 4];
       numbers.fill(0); // numbers  => [0, 0, 0, 0]
       const zeros = Array(3).fill(0); // [0, 0, 0]
```

Індекс елемента

Теорія: Масиви JavaScript

Елементи в масиві нумеруються, починаючи з нуля. Цей номер називається **індексом** елемента. Перший елемент має індекс 0, другий має індекс 1 і так далі. Ви можете отримати доступ до окремих елементів у масиві за допомогою позначення в квадратних дужках.

```
const fruits = ['Apple', 'Banana'];
fruits[0]; // Apple
fruits[1]; // Banana
// The index of an array's last element is always one less than the length of the array.
fruits[fruits.length - 1]; // Banana
// Using a index number larger than the array's length
fruits[99]; // undefined
const shopping = ['bread', 'milk', 'cheese', 'hummus', 'noodles'];
console.log(shopping[0]); // returns "bread"

// You can also modify an item in an array by giving a single array item a new value. Try this:
shopping[0] = 'tahini';
console.log(shopping); // shopping will now return [ "tahini", "milk", "cheese", "hummus", "noodles" ]
```

Довжина масиву

Теорія: Масиви JavaScript

Загальна кількість елементів масиву зберігається у його властивості **length**:
Ви можете дізнатися довжину масиву за допомогою властивості **length**:
`let customArr = [6, true, 'String'];` **`console.log(customArr.length); // 3`**

Властивість length – не кількість елементів масива, а останній індекс + 1.

```
let arrLength = [];
arrLength[1000] = true;
console.log(arrLength[1000]);
console.log(arrLength.length);
console.log(arrLength[3]); // undefined
```

Розмір масива може збільшуватись або зменшуватись. **При зменшенні length масив скорочується.** Цей процес **незворотній**:

```
arrLength.length = 2;
console.log(arrLength[3]);
arrLength.length = 1000;
console.log(arrLength[1000]); // undefined
```

У масивах можуть зберігатись елементи будь-якого типу

Теорія: Масиви JavaScript

У масивах можуть зберігатись елементи будь-якого типу. Наприклад:

```
// різні типи значень
let arr = [
 'Яблуко',
 { name: 'Микола' },
 true,
 function() { alert('привіт'); }
];
// отримати елемент з індексом 1 (об'єкт) та вивести його властивість name
alert( arr[1].name ); // Микола
// отримати елемент з індексом 3 (функція) та виконати її
arr[3](); // привіт
```

Кома в кінці

Теорія: Масиви JavaScript

Список елементів масиву, як і список елементів об’єкта може закінчуватись **комою**:

```
let fruits = [
 "Яблуко",
 "Апельсин",
 "Слива",
];
```

Кома в кінці рядка спрощує процес додавання/видалення елементів, тому що всі рядки стають однотипними.

Останній елемент масиву

Теорія: Масиви JavaScript

Ми можемо явно обчислити індекс останнього елемента, а потім отримати до нього доступ: fruits[fruits.length - 1].

```
let fruits = ["Яблуко", "Апельсин", "Слива"];

alert( fruits[fruits.length-1] ); // Слива
```

Є коротший синтаксис: fruits.at(-1):

```
let fruits = ["Яблуко", "Апельсин", "Слива"];

// те саме, що й fruits[fruits.length-1]

alert( fruits.at(-1) ); // Слива
```

Інакше кажучи, arr.at(i):

- те саме, що й arr[i], якщо i >= 0.
- для від’ємних значень і він шукає елемент відступаючи від кінця масиву.

Багатовимірні масиви

Теорія: Масиви JavaScript

Масив всередині масиву називається **багатовимірним** масивом. Ви можете отримати доступ до елемента всередині масиву, який сам знаходиться всередині іншого масиву, об’єднавши два набори квадратних дужок. Наприклад, щоб отримати доступ до одного з елементів у масиві, який є третім елементом у масиві [0, 1, 2]:

```
const random = ['tree', 795, [0, 1, 2]];

random[2][2];
```

Метод **array.flat([depth])** створює новий масив шляхом рекурсивного зведення елементів, які є масивами, до певної глибини. Необов'язковий аргумент глибини за умовчанням дорівнює 1.

```
const arrays = [0, [1, 3, 5], [2, 4, 6]];

const flatArray = arrays.flat();


flatArray; // [0, 1, 3, 5, 2, 4, 6]
```

Цикли

Теорія: Масиви JavaScript

При написанні скриптів часто постає завдання зробити однотипні дії багато разів.

Наприклад, вивести товари зі списку один за одним чи просто запустити один і той же код для кожного числа від 1 до 10.

Цикл – це спосіб повторити один і той же код кілька разів.

Одне виконання циклу називається **ітерацією**.

Цикл while

Теорія: Масиви JavaScript

Цикл while має такий синтаксис:

```
while (умова) {
 // код
 // так зване "тіло циклу"
}
```

Доки умова є вірною, виконується код із тіла циклу.

Наприклад, цикл нижче виводить і поки і < 3:

```
let i = 0;
while (i < 3) { // показується 0, далі 1, потім 2
 alert( i );
 i++;
}
```

Якщо тіло циклу має тільки одну операцію

Фігурні дужки не потрібні для однорядкової операції

Якщо тіло циклу має тільки одну операцію, ми можемо опустити фігурні дужки {…}:

```
let i = 3;
while (i) alert(i--);
```

Цикл “do…while”

Теорія: Масиви JavaScript

Перевірка умови може бути переміщена нижче тіла циклу використовуючи do..while синтаксис:

```
do {
 // тіло циклу
} while (умова);
```

Цикл спочатку виконує тіло, а потім перевіряє умову, і поки умова є true, цикл виконується знову і знову.

Цикл do вираз while (умова) виконується мінімум 1 раз.

```
do {
        i += 1;
        console.log(i);
} while (i < 5);
```

Будь-який вираз або змінна може бути умовою циклу

Теорія: Масиви JavaScript

Будь-який вираз або змінна може бути умовою циклу, а не тільки порівняння (a < 5 чи b !== 10).
Умова виконується і конвертується у логічне значення.
Наприклад, коротший спосіб написання while (i != 0) відповідає while (i):

```
let i = 3;
while (i) { // коли і буде 0, умова стане невірною, і цикл зупиниться
 alert( i );
 i--;
}
```

Наприклад:

```
let i = 0;
do {
 alert( i );
 i++;
} while (i < 3);
```

Цю форму синтаксису слід використовувати лише тоді, коли ви хочете, щоб тіло циклу виконалось хоча б один раз, незалежно від умови.

Цикл for

Теорія: Масиви JavaScript

Цикл for є часто використовуваним циклом.

```
for (початок; умова; крок) {
 // ... тіло циклу ...
}
```

Цикл нижче виконує alert(i) для і від 0 до 3 (але не включаючи це число 3)

```
for (let i = 0; i < 3; i++) { // показується 0, далі 1, потім 2
 alert(i);
}
```

- початок let i = 0 Виконується один раз, при вході в цикл.
- умова i < 3 Перевіряється перед кожною ітерацією циклу. Якщо умова невірна, цикл зупиняєтья.
- тіло alert(i) Виконується знову і знову, поки умова є правдивою (true).
- крок i++ Виконується після тіла на кожній ітерації, але перед перевіркою умови.

Загальний алгоритм циклу

Теорія: Масиви JavaScript

Загальний алгоритм циклу працює так:

Цикл for повторює дію, поки не відбудеться подія завершення цикла:

for ([початок]; [умова]; [крок]) вираз

- Виконується вираз початок, якщо вказано. Ініціалізує один або кілька лічильників. Також використовується для визначення змінних.
- Якщо умова істина, то виконується вираз, якщо хибна, цикл for завершується, якщо пропущена, то вважається істина.
- Для виконання кількох виразів використовується блок-вираз { ... }.
- Обновлюється крок, якщо існує, і знову перевіряється умова.

```
for (var i = 0; i<10;i++) {
                console.log(i);
}
```

Вбудоване оголошення змінної

Теорія: Масиви JavaScript

В цьому прикладі всередині циклу оголошена змінна і, яка виконує функцію лічильника. Це так зване **«вбудоване»** оголошення змінної. Такі змінні доступні лише всередині циклу.

```
for (let i = 0; i < 3; i++) {
 alert(i); // 0, 1, 2
}
alert(i); // помилка, немає такої змінної
```

Замість оголошення нової змінної, ми можемо використовувати наявну:

```
let i = 0;
for (i = 0; i < 3; i++) { // використовуємо існуючу змінну
 alert(i); // 0, 1, 2
}
alert(i); // 3, змінна доступна, тому що вона оголошена поза циклом
```

Пропуск частин в “for”

Теорія: Масиви JavaScript

Будь-яку частину for можна пропустити. Наприклад, ми можемо опустити початок, якщо нам не потрібно нічого робити перед стартом циклу.

```
let i = 0; // ми вже маємо оголошену змінну і присвоєне значення
for (; i < 3; i++) { // немає необхідності в "початку"
 alert( i ); // 0, 1, 2
}
```

Ми також можемо видалити частину крок:

```
let i = 0;
for (; i < 3;) {
 alert( i++ );
}
```

Це робить цикл ідентичним до while (i < 3).

Можна взагалі забрати все, отримавши нескінченний цикл:

```
for (;;) {
 // буде вічно повторюватися
}
```

Переривання циклу: “break”

Теорія: Масиви JavaScript

Зазвичай, цикл завершується, коли умова стає false. Але ми можемо в будь-який момент вийти з циклу, використавши спеціальну директиву break.
Наприклад, наступний код запитує в користувача число доти, поки користувач їх вводить. Після того, як користувач не ввів число — цикл завершується (директивою “break”) і виводить суму чисел:

```
let sum = 0;
while (true) {

 let value = +prompt("Введіть число", '');
 if (!value) break; // (*)
 sum += value;
}
alert( 'Сума: ' + sum );
```

Директива break спрацьовує тоді, коли користувач вводить порожній рядок або скасовує введення. Ця директива негайно завершує виконання циклу і передає контроль наступному рядку за циклом, тобто на alert.

Продовження з наступної ітерації

Теорія: Масиви JavaScript

Директива **continue** - це “полегшена версія” break. Вона не зупиняє весь цикл. Натомість вона зупиняє поточну ітерацію і починає виконання циклу спочатку з наступної ітерації .
Її зручно використовувати коли закінчили з поточною ітерацією і хочемо продовжити з наступної.
Цикл нижче використовує continue, щоб вивести лише непарні значення:

```
for (let i = 0; i < 10; i++) {

 // якщо умова справджується, тоді пропускаємо решту тіла циклу і починаємо з наступної ітерації
 if (i % 2 == 0) continue;

 alert(i); // 1, потім 3, 5, 7, 9
}
```

Для парних значень змінної i, директива continue зупиняє виконання тіла циклу і передає контроль наступній ітерації в for (в цьому випадку це буде наступне число). Таким чином функція alert викликається лише для непарних значень змінної i.
Директива continue допомагає зменшити рівень вкладеності

Мітки для break/continue

Теорія: Масиви JavaScript

Деколи нам потрібно вийти з кількох вкладених циклів.
Наприклад, в коді нижче є 2 цикли, які проходяться по змінних i та j, і запитують в користувача координати (i, j) від (0,0) до (2,2):

```
for (let i = 0; i < 3; i++) {
 for (let j = 0; j < 3; j++) {
   let input = prompt(`Значення в координатах (${i},${j})`, '');
   // що якщо ми хочемо вийти звідси відразу до 'Готово!' (в функцію alert нижче)?
 }
}
alert('Готово!');
```

Нам потрібен спосіб зупинити ці два цикли, якщо користувач скасує введення.
Звичайний break після input перерве лише внутрішній цикл, а нам цього недостатньо. Ось тут нам стануть в пригоді мітки для циклів!
Мітка складається з ідентифікатора та двокрапки перед циклом:

```
labelName: for (...) {
 ...
}
```

Найближчий цикл з заданою міткою

Теорія: Масиви JavaScript

Вираз break <labelName> в циклі нижче шукає найближчий цикл з заданою міткою і переходить в його кінець:

```
outer: for (let i = 0; i < 3; i++) {
 for (let j = 0; j < 3; j++) {
   let input = prompt(`Значення в координатах (${i},${j})`, '');
   // якщо порожній рядок або Скасувати, тоді вихід з обох циклів
   if (!input) break outer; // (*)
   // зробити щось із значенням...
 }
}
alert('Готово!');
```

В коді вище, вираз break outer шукає зверху мітку outer і переривает цикл, позначений цією міткою. Тож виконання коду перейде з місця переривання циклу (позначене (*)) до функції alert('Готово!').

Мітку можна перемістити в новий рядок:

```
outer:
for (let i = 0; i < 3; i++) { ... }
```

Також мітками можна використовувати з директивою continue. В такому разі, виконання коду перестрибне на наступну ітерацію поміченого циклу.

Міткам не дозволено “стрибати” будь-де

Теорія: Масиви JavaScript

Ми не можемо використовувати мітки, щоб стрибати в довільне місце в коді.
Наприклад, ось таке неможливо зробити:

```
break label; // стрибнути в мітку label нижче (не спрацює)
label: for (...)
```

Директива break повинна бути всередині блоку з кодом. Технічно, підійде навіть іменований блок.
Наприклад:

```
label: {
 // ...
 break label; // працює
 // ...
}
```

…Однак, 99.9% часу break використовується всередині циклів.
Виклик continue можливий лише всередині циклу.

Методи масиву

Теорія: Масиви JavaScript

- Метод push() використовується для вставки елемента в кінець масиву.

```
const numbers = [1, 22, 123, 0, 15, 9, 88, 123];
console.log(numbers.push(45)); // 9
```

- Метод pop видаляє останній елемент масиву та повертає видалений елемент.

```
console.log(numbers.pop()); // 45
```

- Метод shift схожий на pop. Shift видаляє елемент з початку, тоді як pop видаляє елемент з кінця масиву. Обидва методи повертають видалений елемент.

```
console.log(numbers.shift()); // 1
```

- Метод unshift схожий на метод push. Метод push додає елемент у кінець масиву. Unshift додає елемент на початку масиву. Обидва повертають довжину нового масиву.

```
console.log(numbers.unshift(45)); // 9
```

- indexOf повертає перший індекс елемента, якщо він присутній у масиві. Якщо ні, повертається -1.

```
console.log(numbers.indexOf(1)); // 0
```

- lastIndexOf отримує останній індекс, за яким можна знайти даний елемент. Якщо не знайдено, повертає -1.

Черга

Теорія: Масиви JavaScript

Черга – один з найбільш популярних варіантів використання об’єкта. У комп’ютерних науках так позначають колекцію елементів, яка підтримує дві операції:

- push додає елемент у кінець списку.
- shift видаляє елемент на початку, зміщуючи чергу, таким чином, що 2-й елемент стає 1-м.

Масиви підтримують обидві операції.

На практиці це дуже часто стає у пригоді. Наприклад, черга з повідомлень, які необхідно показувати на екрані.

Стек

Теорія: Масиви JavaScript

Існує також інший варіант використання масивів – структура даних, яка називається стек.

Вона підтримує два типи операцій:
push додає елементи в кінець.
pop видаляє елемент з кінця.

Таким чином нові елементи завжди додаються або видаляються з “кінця”.

Хорошим прикладом стеку є колода карт: нові карти кладуться на верх і беруться теж зверху:

У стеках – останній доданий елемент повертається першим, цей принцип також називають LIFO (з англ. Last-In-First-Out, “останній прийшов – перший пішов”). Для черг ми використовуємо принцип FIFO (з англ. First-In-First-Out, “перший прийшов – перший пішов”).

Масиви в JavaScript можуть працювати як стеки і як черги. Ми можемо додавати/видаляти елементи як на початку, так і у кінці масиву.
В комп’ютерних науках структури даних, які дозволяють це робити, мають назву двобічна черга.

Методи, які працюють з кінцем масиву

Теорія: Масиви JavaScript

Методи, які працюють з кінцем масиву:

pop

Видаляє останній елемент масиву та повертає його:

```
let fruits = ["Яблуко", "Апельсин", "Груша"];

alert( fruits.pop() ); // видаляємо "Груша" та виводимо його

alert( fruits ); // Яблуко,Апельсин
```

Що fruits.pop(), що fruits.at(-1) – обидва повертають останній елемент масиву, але fruits.pop() також змінює масив, видаляючи його.

push

Додає елемент в кінець масиву:

```
let fruits = ["Яблуко", "Апельсин"];

fruits.push("Груша");

alert( fruits ); // Яблуко,Апельсин,Груша
```

Виклик fruits.push(...) рівнозначний fruits[fruits.length] = ...

Методи, які працюють з початком масиву

Теорія: Масиви JavaScript

Методи, які працюють з початком масиву:
shift
Видаляє перший елемент з масиву та повертає його:

```
let fruits = ["Яблуко", "Апельсин", "Груша"];
alert( fruits.shift() ); // видаляємо "Яблуко" та виводимо його
alert( fruits ); // Апельсин,Груша
```

unshift
Додає елемент в початок масиву:

```
let fruits = ["Апельсин", "Груша"];
fruits.unshift('Яблуко');
alert( fruits ); // Яблуко,Апельсин,Груша
```

Методи push та unshift можуть додавати одразу декілька елементів:

```
let fruits = ["Яблуко"];
fruits.push("Апельсин", "Персик"); // ["Яблуко", "Апельсин", "Персик"]
fruits.unshift("Ананас", "Лимон");
// ["Ананас", "Лимон", "Яблуко", "Апельсин", "Персик"]
alert( fruits );
```

Метод indexOf

Теорія: Масиви JavaScript

Метод indexOf повертає перший індекс заданого елемента, якщо він присутній у масиві. Якщо ні, повертається -1. Щоб перевірити, чи присутній елемент у масиві, є метод include().

```
var array = [2, 9, 9];
array.indexOf(2); // 0
array.indexOf(7); // -1
array.indexOf(9, 2); // 2
array.indexOf(2, -3); // 0
```

lastIndexOf отримує останній індекс, за яким можна знайти даний елемент. Якщо не знайдено, повертає -1. Він може приймати другий параметр, який використовується як fromIndex. Пошук починається назад від наданого індексу.

```
const numbers = [1, 22, 123, 0, 15, 9, 88, 123, 0, 45];
console.log(numbers.lastIndexOf(2)); // -1
console.log(numbers.lastIndexOf(22)); // 1
console.log(numbers.lastIndexOf(123, 4)); // 2
console.log(numbers.lastIndexOf(0, 6)); // 3
console.log(numbers.lastIndexOf(1, 1)); // 0
```

Метод Includes

Теорія: Масиви JavaScript

Метод Includes перевіряє, чи містить масив елемент, і повертає true або false.

```
const array1 = [1, 2, 3];
console.log(array1.includes(3)); // true
console.log(array1.includes(4)); // false
const pets = ["cat", "dog", "bat"];
console.log(pets.includes("cat")); // true
console.log(pets.includes("at")); // false
```

// Метод include також приймає index як другий параметр. Другий параметр змушує метод include перевіряти значення в масиві з індексом, більшим або рівним наданому індексу.

```
let example = ["a", "b", "c"]
example.includes("b", 3); // false
example.includes("b", 100); // false
example.includes("b", 1); // true
```

Метод slice

Теорія: Масиви JavaScript

Метод slice використовується для вирізання певної позиції масиву. Цей метод приймає два аргументи, початок і кінець.

- Аргумент «початок» визначає, з якого індексу слід починати зрізання,
- Аргумент «кінець» визначає, на якому індексі слід припинити зрізання.

Він не замінює існуючий масив і повертає новий нарізаний масив.

```
var students = ["Tom", "Mary", "Ann", "John", "Bob"];

var start_index = 2, end_index = 4;
students.slice(start_index, end_index);

//Console Output (2) ["Ann", "John"]
```

Метод splice

Теорія: Масиви JavaScript

Для видалення певного індексу масиву використовується інструкція **delete**
var names = ["John", "Ann", "Cat"];
delete names[1];
Після видалення кількість елементів масиву не змінюється, а значення видаленого елемента становить undefined.
names[1] //Console output undefined

Щоб повністю видалити елемент з індексом, можна використати метод **splice**().
Перший аргумент методу - індекс, який потрібно видалити, а другий - кількість елементів, які потрібно видалити. Метод повертає видалені елементи масиву.

```
names.splice(1,1);
console.log("After deleting item of index 1 ", names);
//Console output ["John", "Cat"]
```

Оператор поширення(...)

Теорія: Масиви JavaScript

Ви можете видаляти елементи з масиву, комбінуючи оператор поширення(...) з літералом масиву.

```
const names = ['Batman', 'Catwoman', 'Joker', 'Bane'];
const fromIndex = 1;
const removeCount = 2;

const newNames = [
 ...names.slice(0, fromIndex),
 ...names.slice(fromIndex + removeCount)
];

newNames; // => ['Batman', 'Bane']
```

Деструктуризація

Теорія: Масиви JavaScript

Деструктуризація - це техніка, яка дозволяє розпакувати значення з масивів або об’єктів в окремі змінні. Цей процес передбачає розбиття складних структур даних на простіші частини, що полегшує роботу з ними. Деструктуризація спрощує процес вилучення значень із масиву до одного рядка коду:

```
const hobbies = ["Reading", "Coding", "Hiking"];
const [firstHobby, secondHobby, thirdHobby] = hobbies;
```

Можна ігнорувати певні елементи, пропускаючи їх із шаблону деструктуризації:

```
const [firstHobby, , thirdHobby] = hobbies;
```

Приклад обміну значеннями елементів масиву за допомогою веструктуризації:

```
const swapElements = (myArray, index1, index2) => {
  [myArray[index1], myArray[index2]] = [myArray[index2], myArray[index1]];
};
```

Метод concat

Теорія: Масиви JavaScript

Метод concat повертає масив, об’єднаний з іншим масивом або значеннями.

```
const array1 = ["a", "b", "c"];
const array2 = ["d", "e", "f"];
const array3 = array1.concat(array2);
console.log(array3); // ["a", "b", "c", "d", "e", "f"]
// Можна об’єднати значення з масивом.
const letters = ["a", "b", "c"];
const alphaNumeric = letters.concat(1, [2, 3]);
console.log(alphaNumeric); // ["a", "b", "c", 1, 2, 3]
// Або об’єднувати вкладені масиви.
const num1 = [[1]];
const num2 = [2, [3]];
const numbers = num1.concat(num2);
console.log(numbers); // [[1], 2, [3]]
```

Метод join

Теорія: Масиви JavaScript

Метод join створює та повертає рядок шляхом об’єднання всіх елементів у масиві.
Метод join може приймати необов’язковий параметр,.join([**seperator**]). За замовчуванням метод використовує кому (,) як роздільник.

```
const fruits = ["Apple", "Banana", "Raspberry"];
console.log(fruits.join()); // Apple,Banana,Raspberry
console.log(fruits.join("")); // AppleBananaRaspberry
console.log(fruits.join("|")); // Apple|Banana|Raspberry
```

Метод toString

Теорія: Масиви JavaScript

Метод toString повертає рядок з елементів масиву

```
const sample = [1, 2, "Parwinder", true, "Hello", 78];
console.log(sample.toString()); // 1,2,Parwinder,true,Hello,78
```

Робота із масивами

Створення програми сортування масиву чисел

Синтаксис [...mutrableNumbers]

Метод sort(): за замовчуванням сортує елементи як рядки, тому для чисел обов'язково потрібно передавати порівняльну функцію (a, b):a - b сортує за зростанням (якщо результат від'ємний, a йде перед b).b - a сортує за спаданням.

Синтаксис [...mutableNumbers]: створює копію масиву перед сортуванням. Це важливо, оскільки метод sort() мутує (змінює) оригінальний масив, для якого він викликаний.

```
// Початковий масив чисел
const numbers = [12, 5, 8, 130, 44, 5, 17, 2, -5, 0];

// 1. створює копію масиву
const mutableNumbers = numbers;

// 2. СОРТУВАННЯ
// Сортування масиву за зростанням (від меншого до більшого)
const sortedAscending = [...mutableNumbers].sort((a, b) => a - b);
```

Сортування масиву чисел

Практика: Масиви JavaScript

```
// Початковий масив чисел
const numbers = [12, 5, 8, 130, 44, 5, 17, 2, -5, 0];

// 1. створює копію масиву
const mutableNumbers = numbers;

// 2. СОРТУВАННЯ
// Сортування масиву за зростанням (від меншого до більшого)
const sortedAscending = [...mutableNumbers].sort((a, b) => a - b);

// Сортування масиву за спаданням (від більшого до меншого)
const sortedDescending = [...mutableNumbers].sort((a, b) => b - a);
// Виведення результатів у консоль
console.log("Оригінальний масив:", numbers);
console.log("копія масиву", mutableNumbers);
console.log("Відсортований за зростанням:", sortedAscending);
console.log("Відсортований за спаданням:", sortedDescending);
```

ВПРАВИ



Вправи до цього заняття знаходяться за посиланням
https://github.com/coachjanus/frontend-dev

Посилання на довідники



Консорціум Всесвітнього павутиння https://www.w3.org/

ECMAScript https://ru.wikipedia.org/wiki/ECMAScript

ECMA http://www.ecma-international.org/

Standard ECMA-262 http://www.ecma-international.org/publications/standards/Ecma-262.htm

JavaScript Web APIs:

- https://www.w3.org/standards/webdesign/script
- https://developer.mozilla.org/en-US/docs/Glossary/JavaScript

Дякую за увагу!

До наступної зустрічі!

QALight

www.qalight.ua
info@qalight.ua